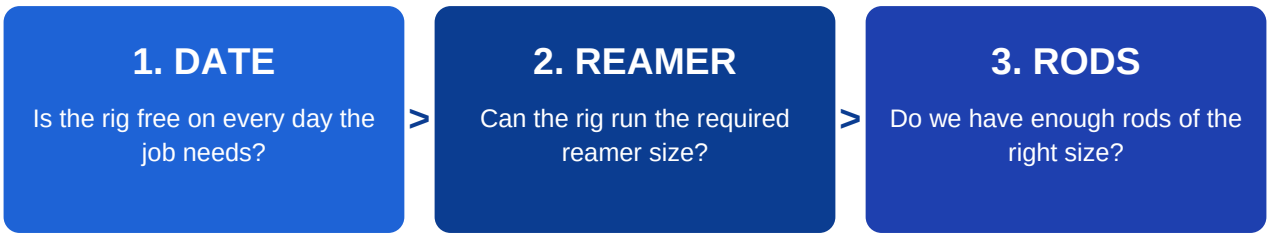
RIG PLANNING APP

Building it from scratch in Python
with an Excel database

A complete, hands-on tutorial

The Three Sequential Checks



If any check fails, the remaining checks are skipped.

Python + pandas + openpyxl

Complete tutorial with full source code

Table of Contents

Table of Contents 2

1. Introduction 4

1.1 What you'll be able to do after this tutorial . . . 4

1.2 How to read this PDF . . . 4

2. What We Are Building 5

2.1 The folder layout . . . 5

3. Setting Up Python From Scratch 6

3.1 Install Python . . . 6

3.2 Create a project folder and a virtual environment . . . 6

3.3 Install the libraries we need . . . 6

3.4 A quick sanity check . . . 7

4. The Excel Database Design 8

4.1 Why split data across five sheets? . . . 8

4.2 The five sheets, in full . . . 8

4.3 Building the workbook with Python . . . 9

5. Reading Excel With pandas 11

5.1 Loading all sheets at once . . . 11

5.2 Filtering a DataFrame . . . 11

5.3 Excel dates vs Python dates . . . 11

6. The Database Loader Class 13

6.1 What the class needs to do . . . 13

6.2 Writing the constructor . . . 13

6.3 A few helpers . . . 14

7. Modeling Jobs With Dataclasses 15

7.1 The Job class . . . 15

7.2 The PlanResult class . . . 15

8. Check 1 - The Date Check 17

8.1 Reading the code . . . 17

8.2 Worked example . . . 17

9. Check 2 - The Reamer Check 18

9.1 Reading the code . . . 18

9.2 Worked example . . . 18

10. Check 3 - The Rod Check 19

10.1 Three sub-checks, in order . . . 19

10.2 Why math.ceil? . . . 19

10.3 Worked example . . . 20

11. The Sequential Engine 21

11.1 Why the three early returns? . . . 21

11.2 Reading a result . . . 21

12. Greedy Job Assignment 22

12.1 Why greedy? . . . 22

12.2 Updating state mid-loop . . . 22

13. Putting It All Together 23

13.1 Run it . . . 23

14. Reading the Output 25

14.1 Job J-201 - one rejection, one win . . . 25

14.2 Job J-202 - a tight rod check . . . 25

14.3 Job J-203 - small but easy . . . 25

14.4 A bird's-eye view . . . 25

15. Extending the System 26

15.1 Persist the plan back to Excel . . . 26

15.2 Add priorities . . . 26

15.3 Consider crew availability . . . 26

15.4 Show alternatives, not just the first match . . . 26

15.5 Run "what-if" scenarios . . . 26

15.6 A graphical front-end . . . 27

Appendix A - Full create_database.py 28

Appendix B - Full rig_planner.py 32

Appendix C - Glossary 39

1. Introduction

This tutorial walks you through building a complete **rig planning application** in Python, from a blank folder all the way to a working tool that reads an Excel database and decides which rig can take which job. The application solves a real, concrete scheduling problem: given a list of upcoming jobs, each with a start date, a required reamer size, and a hole length, the program must pick a rig that can do the job.

The logic follows three checks in a strict order. If any check fails, the rig is rejected and the remaining checks are skipped for that rig. This is called **short-circuit evaluation**, and it mirrors the way an experienced planner thinks: there is no point checking rod inventory if the rig is already booked on the day the job starts.

The three checks, in order

Order	Check	What it asks	If it fails...
1	Date	Is the rig free every day the job needs?	skip checks 2 and 3 for this rig
2	Reamer	Can the rig physically run the required reamer size?	skip check 3 for this rig
3	Rods	Do we own enough rods of the right size?	the rig is rejected

Why this order? Date conflicts are absolute - no amount of rod inventory can free a busy rig. Reamer capability is a hardware limit - it cannot be fixed at planning time. Rod count is the most flexible (we can sometimes borrow or order more), so we check it last. The order goes from the hardest constraint to the softest.

1.1 What you'll be able to do after this tutorial

- Open Python from scratch and install the libraries we need.
- Design a clean Excel workbook that doubles as a database.
- Read Excel sheets into Python using **pandas**.
- Model real-world entities (Rigs, Jobs, Rods) using **dataclasses**.
- Write small, focused functions for each rule.
- Combine those functions into a sequential decision engine.
- Print clean, auditable output that explains every decision.

1.2 How to read this PDF

Each chapter introduces one idea, shows the Python code for it, and ends with a short explanation of *why* the code is written the way it is. Code blocks look like this:

```
print("Hello, rigs!")
```

Important reminders and warnings appear in orange boxes like the one above. The full source code of every file is collected in the appendix at the end so you can copy it into your editor in one go.

2. What We Are Building

Imagine the planning office has five drilling rigs (RIG-A to RIG-E). Each rig has its own schedule, its own list of reamer sizes it is certified to run, and its own maximum hole length. The yard holds a limited number of drill rods in three different diameters.

Tomorrow morning, three new jobs arrive on the planner's desk:

Job	Start date	Duration	Reamer size	Hole length
J-201	2026-06-01	2 days	12.25 in	1 800 m
J-202	2026-06-05	3 days	17.5 in	1 080 m
J-203	2026-06-08	1 day	8.5 in	900 m

The planner's question is simple to ask but tedious to answer by hand: **which rig should I assign to each job?** The program we are about to build answers this question in seconds, and tells us exactly *why* any rig was rejected. That "why" matters - if a rig fails on rods, we know to order more rods. If it fails on reamer capability, we know to send a different crew.

2.1 The folder layout

By the end of this tutorial your project folder will contain three files - one Excel workbook and two Python scripts:

```
rig_planner/
  rig_database.xlsx        <- our "database"
  create_database.py       <- one-off script that builds the .xlsx
  rig_planner.py           <- the planning program itself
```

Keeping the database creation in its own script (instead of typing data into Excel by hand) means anyone can regenerate the workbook from scratch by running one command. That is great for testing and for keeping the example reproducible.

3. Setting Up Python From Scratch

If you have never run Python before, this chapter gets you to a working installation. If you already have Python and pip, skip ahead to section 3.3.

3.1 Install Python

Go to **python.org/downloads** and download the latest stable release for your operating system. During installation on Windows, make sure to tick the box that says "Add Python to PATH". On macOS and Linux the installer puts Python on PATH for you.

Verify the install by opening a terminal (Command Prompt on Windows, Terminal on macOS or Linux) and typing:

```
python --version
```

You should see something like `Python 3.12.1`. Any version from 3.10 onwards will run the code in this tutorial.

3.2 Create a project folder and a virtual environment

A **virtual environment** is a private Python copy that belongs to your project. It keeps the libraries we install here separate from the rest of your computer. Create one with:

```
mkdir rig_planner
cd rig_planner
python -m venv .venv

# Activate it:
#   Windows:  .venv\Scripts\activate
#   macOS:    source .venv/bin/activate
#   Linux:    source .venv/bin/activate
```

After activation, your prompt will show `(.venv)` at the start. Any `pip install` commands you run now will install into this private environment only.

3.3 Install the libraries we need

This project depends on three libraries:

Library	What it does	Why we need it
pandas	Reads and writes tabular data	Loading the Excel sheets
openpyxl	The actual Excel file engine	pandas uses it under the hood for .xlsx
reportlab	Creates PDF reports	Optional - only if you want to export plans to PDF

Install all three with one command:

```
pip install pandas openpyxl reportlab
```

> **Tip:** if `pip` is not recognised, try `python -m pip install ...` instead. This form works on every operating system.

3.4 A quick sanity check

Create a file called `hello.py` with this content, then run it with `python hello.py`:

```
import pandas as pd
df = pd.DataFrame({"rig": ["RIG-A", "RIG-B"], "jobs": [3, 1]})
print(df)
```

If you see a small table printed in the terminal, your environment is ready and we can move on to designing the Excel database.

4. The Excel Database Design

We will treat one Excel workbook as our entire database. Inside the workbook, each **sheet** is a table and each **column** is a field. That is exactly how a small SQL database is laid out - we are just using a different tool for storage.

4.1 Why split data across five sheets?

It would be tempting to put everything on one giant sheet, but that creates two problems: data gets repeated (the same rig name appears in many rows), and the file becomes very hard to update without introducing inconsistencies. Splitting by *entity* keeps the data **normalised**: every fact lives in exactly one place.

Sheet	Holds...	Example row
Rigs	the master list of rigs and their depth limi	RIG-A, Field North, 3500 m, Active
RigSchedule	every day each rig is already booked	RIG-A, 2026-06-01, Job-100
ReamerCapability	which reamer sizes each rig can run	RIG-A, 12.25 in
Rods	rod inventory by size	4.5 in, 9 m/rod, 250 available
ReamerRodMapping	which rod size pairs with which reamer	12.25 in reamer -> 4.5 in rod

4.2 The five sheets, in full

Sheet 1 - Rigs

RigName	Location	MaxHoleLength_m	Status
RIG-A	Field North	3500	Active
RIG-B	Field North	2500	Active
RIG-C	Field South	4000	Active
RIG-D	Field South	1800	Active
RIG-E	Field East	3000	Active

Sheet 2 - RigSchedule

Each row marks one day the rig is **busy**. If a rig has no row for a given date, it is implicitly free that day. Storing only busy days keeps the sheet small and easy to maintain.

RigName	BusyDate	Job
RIG-A	2026-06-01	Job-100
RIG-A	2026-06-02	Job-100
RIG-A	2026-06-03	Job-100
RIG-B	2026-06-05	Job-101
RIG-B	2026-06-06	Job-101
RIG-C	2026-06-01	Job-102

...	...	...

Sheet 3 - ReamerCapability

Each row is one (rig, reamer-size) pair the rig is *certified* to run. A rig with three rows can run three reamer sizes; a rig with one row can only run one. This format is called a **many-to-many join table** - it is the standard way to express "X can do Y" relationships.

RigName	ReamerSize_in
RIG-A	8.5
RIG-A	12.25
RIG-A	17.5
RIG-B	8.5
RIG-B	12.25
RIG-D	8.5
...	...

Sheet 4 - Rods

The rod inventory. Each row gives a rod size, how long one rod is, and how many are currently in stock.

RodSize_in	LengthPerRod_m	AvailableCount
3.5	9	400
4.5	9	250
5.5	9	120

Sheet 5 - ReamerRodMapping

The bridge between a reamer size and the rod size that fits it. Without this table the program would have no way to translate "this job uses a 12.25 in reamer" into "so we need 4.5 in rods".

ReamerSize_in	RequiredRodSize_in
8.5	3.5
12.25	4.5
17.5	5.5

4.3 Building the workbook with Python

Instead of typing the data into Excel by hand we will write a one-off Python script. This has two big advantages: anyone can regenerate the database, and the data is version-controlled along with the code.

Create a file called `create_database.py` and start with the imports:

```
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from datetime import date

wb = Workbook()                 # creates a new in-memory workbook
```

A fresh workbook starts with one empty sheet. We rename it to `Rigs`, append a header row, then a data row for every rig:

```
ws1 = wb.active
ws1.title = "Rigs"
ws1.append(["RigName", "Location", "MaxHoleLength_m", "Status"])
for r in [
    ["RIG-A", "Field North", 3500, "Active"],
    ["RIG-B", "Field North", 2500, "Active"],
    ["RIG-C", "Field South", 4000, "Active"],
    ["RIG-D", "Field South", 1800, "Active"],
    ["RIG-E", "Field East",  3000, "Active"],
]:
    ws1.append(r)
```

The same pattern (create a sheet, append a header, append rows) is used for every other sheet. The full script is reproduced in **Appendix A**. After you run it, you will have `rig_database.xlsx` in your folder.

```
python create_database.py
```

Open it in Excel and have a look. Seeing the data with your own eyes makes the rest of the tutorial much easier to follow. You can also edit the values directly - the planner reads the file every time it runs, so any change is picked up automatically.

5. Reading Excel With pandas

The pandas library lets us load an Excel sheet into a Python object called a **DataFrame**. A DataFrame behaves like a table: it has named columns, it has a row index, and you can filter and join it with one-liners. We will use just three pandas features:

- `pd.read_excel(...)` - loads a sheet (or every sheet) into a DataFrame
- `df.loc[condition, "column"]` - filters rows and picks one column
- `pd.to_datetime(...).dt.date` - converts Excel timestamps into plain Python dates

5.1 Loading all sheets at once

Passing `sheet_name=None` to `read_excel` returns a dictionary that maps each sheet name to its DataFrame. One line gives us everything:

```
import pandas as pd

sheets = pd.read_excel("rig_database.xlsx", sheet_name=None)
rigs_df       = sheets["Rigs"]
schedule_df   = sheets["RigSchedule"]
capability_df = sheets["ReamerCapability"]
rods_df       = sheets["Rods"]
mapping_df    = sheets["ReamerRodMapping"]
```

5.2 Filtering a DataFrame

Imagine we want every day RIG-A is busy. With pandas this reads almost like English:

```
busy_dates = schedule_df.loc[
    schedule_df["RigName"] == "RIG-A",  # condition
    "BusyDate"                            # column to keep
]
print(list(busy_dates))
# -> [datetime(2026,6,1), datetime(2026,6,2), datetime(2026,6,3)]
```

The condition on the left selects the rows; the column name on the right picks which field we want from those rows. We could chain more conditions with the `&` operator (AND) and `|` (OR), but for this project the single-condition form is enough.

5.3 Excel dates vs Python dates

When pandas reads a date column from Excel, it stores each value as a `pandas.Timestamp` (which is essentially a Python `datetime`). For our "is this day in the busy set?" check we want plain `date` objects, because that makes set membership easy to reason about. The conversion is one line:

```
schedule_df["BusyDate"] = pd.to_datetime(
    schedule_df["BusyDate"]
).dt.date
```

Why is this important? If we forget the conversion, `date(2026,6,1)` from our job will not match `Timestamp('2026-06-01')` from the schedule, because they are different types - even though

they look the same when printed. Bugs like this are notoriously hard to spot.

6. The Database Loader Class

We could keep all the DataFrames as loose variables, but that becomes hard to manage as the project grows. Wrapping them in a small **class** gives us a single object we can pass around, and lets us pre-compute fast lookups in the constructor.

6.1 What the class needs to do

- Load every sheet of the workbook (once).
- Normalise the date column.
- Build four quick-lookup structures so the check functions can run in O(1) per query.

The four lookups are:

Lookup	Type	Used by
self.busy	dict: rig -> set of busy dates	the DATE check
self.capable	dict: rig -> set of reamer sizes	the REAMER check
self.reamer_to_rod	dict: reamer size -> rod size	the ROD check
self.rod_inventory	dict: rod size -> {length, available}	the ROD check

6.2 Writing the constructor

We will build the class in three small chunks so each piece is easy to follow. Begin with the import block and the loading part:

```
from typing import Dict, Set, List
import pandas as pd

class RigDatabase:
    def __init__(self, path: str = "rig_database.xlsx"):
        self.path = path
        sheets = pd.read_excel(path, sheet_name=None)

        self.rigs_df       = sheets["Rigs"]
        self.schedule_df   = sheets["RigSchedule"]
        self.capability_df = sheets["ReamerCapability"]
        self.rods_df       = sheets["Rods"]
        self.mapping_df    = sheets["ReamerRodMapping"]

        # Normalise dates
        self.schedule_df["BusyDate"] = pd.to_datetime(
            self.schedule_df["BusyDate"]).dt.date
```

Now add the busy-day and capability lookups. Each loop walks through every rig and builds a Python **set** of values - sets are perfect because they answer "is X in here?" in constant time.

```
        # 1) Busy days per rig
        self.busy: Dict[str, Set] = {}
        for rig in self.rigs_df["RigName"]:
            self.busy[rig] = set(
                self.schedule_df.loc[
                    self.schedule_df["RigName"] == rig,
                    "BusyDate"])

        # 2) Reamer sizes each rig can run
        self.capable: Dict[str, Set[float]] = {}
        for rig in self.rigs_df["RigName"]:
            self.capable[rig] = set(
                self.capability_df.loc[
                    self.capability_df["RigName"] == rig,
                    "ReamerSize_in"].astype(float))
```

Finally, the rod-mapping and rod-inventory dictionaries. Both are small enough that we just build them with a comprehension and a loop:

```
        # 3) Reamer -> rod size
        self.reamer_to_rod = dict(zip(
            self.mapping_df["ReamerSize_in"].astype(float),
            self.mapping_df["RequiredRodSize_in"].astype(float)))

        # 4) Rod inventory keyed by size
        self.rod_inventory = {}
        for _, row in self.rods_df.iterrows():
            self.rod_inventory[float(row["RodSize_in"])] = {
                "length_per_rod": float(row["LengthPerRod_m"]),
                "available":      int(row["AvailableCount"]),
            }
```

6.3 A few helpers

Two more tiny methods make the rest of the program nicer to read: a property that returns the list of rig names, and a helper that looks up a rig's maximum hole length.

```
    @property
    def rig_names(self) -> List[str]:
        return list(self.rigs_df["RigName"])

    def max_hole_length(self, rig: str) -> float:
        return float(
            self.rigs_df.loc[
                self.rigs_df["RigName"] == rig,
                "MaxHoleLength_m"
            ].iloc[0])
```

Why do we pre-build all these lookups in the constructor? We will call the check functions many times (every job against every rig). If each call re-filtered the DataFrame it would re-scan thousands of rows. Pre-computing once means every later call is essentially free.

7. Modeling Jobs With Dataclasses

A **dataclass** is a Python feature that turns a plain class into a tidy record type. We just declare the fields and Python writes the `__init__`, `__repr__`, and equality logic for us. That keeps our code declarative - the class becomes a short, readable description of what a Job *is*.

7.1 The Job class

```
from dataclasses import dataclass
from datetime import date, timedelta
from typing import List

@dataclass
class Job:
    job_id: str
    start_date: date
    duration_days: int
    reamer_size_in: float
    hole_length_m: float

    @property
    def required_dates(self) -> List[date]:
        return [self.start_date + timedelta(days=i)
                for i in range(self.duration_days)]
```

The `required_dates` property is a convenience that expands a (start_date, duration) pair into the full list of calendar days. A 3-day job starting 2026-06-05 becomes `[2026-06-05, 2026-06-06, 2026-06-07]`. We will use this list directly in the date check.

7.2 The PlanResult class

Every time we try to fit a job to a rig, we want to record what happened: which checks passed, what the failure reason was, and how many rods were reserved. Bundling those fields into another dataclass keeps the engine clean.

```
@dataclass
class PlanResult:
    job_id: str
    rig_name: str
    passed_date: bool = False
    passed_reamer: bool = False
    passed_rods: bool = False
    rods_needed: int = 0
    rod_size_in: float = 0.0
    reason: str = ""

    @property
    def is_assignable(self) -> bool:
        return (self.passed_date
                and self.passed_reamer
                and self.passed_rods)
```

The boolean fields default to `False`. Each check function will flip them to `True` as it passes. The `is_assignable` property is just a shortcut for "all three checks passed".

> **Tip:** the order of fields in a dataclass matters - any field without a default must come before fields with defaults. That is why `job_id` and `rig_name` are at the top.

8. Check 1 - The Date Check

The first and strictest filter. A rig that is busy on any one of the days the job needs is unusable, period. We express the rule in a few lines:

```
from typing import Tuple

def check_date(db: RigDatabase, rig: str, job: Job) -> Tuple[bool, str]:
    clashes = [d for d in job.required_dates
               if d in db.busy[rig]]
    if clashes:
        return False, f"busy on {', '.join(str(d) for d in clashes)}"
    return True, "all required dates free"
```

8.1 Reading the code

`db.busy[rig]` is the set we built in section 6 - it answers "is this date a busy day for this rig?" almost instantly. The list comprehension keeps every date that clashes. If that list is non-empty we return `False` with a human-readable message; otherwise `True`.

Notice that the function returns a tuple `(bool, str)`. The boolean drives the decision, the string drives the audit log. We want both - it is almost free to compute the reason at the same time, and saves the user from wondering "why exactly did this rig fail?".

8.2 Worked example

Take J-201: start 2026-06-01, 2 days. Its required dates are 2026-06-01 and 2026-06-02. The busy set for RIG-A contains both (from Job-100). The comprehension yields `[2026-06-01, 2026-06-02]`, which is truthy, so RIG-A is rejected. The function returns:

```
(False, "busy on 2026-06-01, 2026-06-02")
```

Now check the same job against RIG-B. RIG-B's busy set contains only 2026-06-05 and 2026-06-06 - neither is in the required dates - so the comprehension yields `[]`, which is falsy. The function returns:

```
(True, "all required dates free")
```

Edge case worth noting: if two jobs both want the same rig on adjacent but non-overlapping days, the rig can do both. Our model handles this automatically because we store busy *days*, not busy *jobs*.

9. Check 2 - The Reamer Check

Even if a rig is free on every day we need, we still cannot use it if it is not certified to run the size of reamer the job requires. Like the date check, this is a strict hardware / capability constraint - it cannot be solved at planning time.

```
def check_reamer(db: RigDatabase, rig: str, job: Job) -> Tuple[bool, str]:
    if job.reamer_size_in not in db.capable[rig]:
        return False, (
            f"cannot run {job.reamer_size_in}\" reamer "
            f"(capable: {sorted(db.capable[rig])})")
    return True, f"{job.reamer_size_in}\" reamer supported"
```

9.1 Reading the code

`db.capable[rig]` is a `set` of floats - the sizes that rig is certified to run. Membership test (`in`) is O(1) for sets, so this check is as cheap as it gets.

If the size is missing, we tell the user exactly what the rig *can* do, sorted for readability. This makes the audit log actionable: the planner now knows whether to look for another rig or to find a different reamer.

9.2 Worked example

Take J-202: it needs a 17.5 in reamer. RIG-D is only capable of 8.5 in. The function returns:

```
(False, 'cannot run 17.5" reamer (capable: [8.5])')
```

Now consider RIG-C: capable of 12.25 and 17.5. The check passes and returns `(True, '17.5" reamer supported')`. The next step - the rod check - now runs.

> **Capability vs availability:** capability is about whether a rig *can* do something at all (which never changes between jobs). Availability is about whether it *is free* to do it (which changes constantly). The reamer check is purely capability.

10. Check 3 - The Rod Check

The most involved of the three, because two things must hold at once: (a) the hole must not exceed the rig's depth limit, and (b) we must own enough rods of the right size to reach the bottom.

```
import math

def check_rods(db: RigDatabase, rig: str, job: Job
               ) -> Tuple[bool, str, int, float]:
    # 3a) rig's own depth limit
    if job.hole_length_m > db.max_hole_length(rig):
        return (False,
                f"hole length {job.hole_length_m} m exceeds rig "
                f"limit {db.max_hole_length(rig)} m",
                0, 0.0)

    # 3b) which rod pairs with this reamer?
    rod_size = db.reamer_to_rod.get(job.reamer_size_in)
    if rod_size is None:
        return False, f"no rod mapping for reamer {job.reamer_size_in}\"", 0, 0.0

    inv = db.rod_inventory.get(rod_size)
    if inv is None:
        return False, f"rod size {rod_size}\" not in inventory", 0, rod_size

    # 3c) how many rods does the hole need?
    rods_needed = math.ceil(job.hole_length_m / inv["length_per_rod"])

    if rods_needed > inv["available"]:
        return (False,
                f"need {rods_needed} x {rod_size}\" rods, "
                f"only {inv['available']} available",
                rods_needed, rod_size)

    return (True,
            f"{rods_needed} x {rod_size}\" rods reserved "
            f"(stock {inv['available']})",
            rods_needed, rod_size)
```

10.1 Three sub-checks, in order

The function does three things, also short-circuited:

Sub-check	What it asks
3a Depth limit	is the hole shallower than the rig's max?
3b Mapping	does this reamer size translate to a real rod size?
3c Inventory	do we own enough rods of that size?

10.2 Why math.ceil?

Rods come in fixed lengths (9 m each in our example). For a 1 080 m hole we need `1080 / 9 = 120` rods exactly. But for an 1 800 m hole we need `1800 / 9 = 200` rods exactly - and for 1 085 m we need `1085 / 9 = 120.55...`, which rounds *up* to 121. You can never use half a rod, so we always round up with `math.ceil`.

10.3 Worked example

J-202 - 17.5 in reamer, 1 080 m hole, RIG-A:

- RIG-A max hole length is 3 500 m. 1 080 <= 3 500, depth limit passes.
- Reamer mapping: 17.5 -> 5.5 in rod.
- Rod count needed: ceil(1080 / 9) = 120.
- Stock of 5.5 in rods: 120. We need 120. 120 <= 120, so the check just passes.

The function returns: `(True, '120 x 5.5" rods reserved (stock 120)', 120, 5.5)`. The number 120 and the rod size 5.5 are remembered so the engine can deduct them from inventory once the assignment is confirmed.

11. The Sequential Engine

Now we glue the three checks together. The user's rule is precise: if a check fails, do not run the rest. We implement that with early `return` statements - the classic way to express short-circuit evaluation in Python.

```
def evaluate(db: RigDatabase, rig: str, job: Job) -> PlanResult:
    res = PlanResult(job_id=job.job_id, rig_name=rig)

    # Step 1 - DATE
    ok, why = check_date(db, rig, job)
    res.passed_date = ok
    if not ok:
        res.reason = f"[DATE FAIL] {why}"
        return res                  # short-circuit

    # Step 2 - REAMER
    ok, why = check_reamer(db, rig, job)
    res.passed_reamer = ok
    if not ok:
        res.reason = f"[REAMER FAIL] {why}"
        return res                  # short-circuit

    # Step 3 - RODS
    ok, why, n_rods, rod_size = check_rods(db, rig, job)
    res.passed_rods = ok
    res.rods_needed = n_rods
    res.rod_size_in = rod_size
    res.reason = ("PASS - " + why) if ok else f"[ROD FAIL] {why}"
    return res
```

11.1 Why the three early returns?

Each `return res` exits the function immediately, skipping every check below it. This delivers exactly the behaviour you asked for - "if one got crossed, do not check the rest". As a bonus, it also makes the function fast: a busy rig spends almost no time being evaluated.

11.2 Reading a result

Every call to `evaluate` returns a `PlanResult` object. By looking at its boolean flags, you can tell exactly how far the rig got in the pipeline:

passed_date	passed_reamer	passed_rods	Meaning
False	False	False	stopped at the date check
True	False	False	passed dates, stopped at reamer
True	True	False	passed dates and reamer, stopped at rods
True	True	True	PASS - rig can take the job

Diagnostic value: these flags make it easy to count, later, how many rigs failed at each stage. If the rod check is killing every assignment, you know to look at rod inventory. If the date check fails most often, you have a scheduling pressure problem, not an inventory problem.

12. Greedy Job Assignment

`evaluate` tells us whether ONE rig can do ONE job. We still need the outer loop that walks through every job and tries every rig. We will use a simple **greedy** approach: take the jobs in the order they arrive, and for each job grab the first rig that passes all three checks.

After a rig is chosen, we update the database in memory so the next job sees the consequences: the rig is now busy on those days, and the rod inventory has been reduced.

```
def plan_jobs(db: RigDatabase, jobs: List[Job]):
    output = []
    for job in jobs:
        attempts = []
        winner = None

        for rig in db.rig_names:
            r = evaluate(db, rig, job)
            attempts.append(r)

            if r.is_assignable:
                winner = r
                # Reserve resources so later jobs see them used
                for d in job.required_dates:
                    db.busy[rig].add(d)
                db.rod_inventory[r.rod_size_in]["available"] -= r.rods_needed
                break

        output.append((job, winner, attempts))
    return output
```

12.1 Why greedy?

Greedy is not always optimal - a smarter planner might hold RIG-A back for a later, harder job. But greedy is easy to read, easy to debug, and good enough for most real planning desks. The full optimal version is a known operations-research problem (the *assignment problem*) and can be added later without changing the check functions.

12.2 Updating state mid-loop

The two lines that update state are crucial:

```
for d in job.required_dates:
    db.busy[rig].add(d)
db.rod_inventory[r.rod_size_in]["available"] -= r.rods_needed
```

After these run, our in-memory database reflects the new schedule. If we did not do this, two jobs could both claim the last 120 rods. Notice that we do NOT save the changes back to the Excel file - the file is the source of truth, and we only modify a working copy. (If you want to persist the plan, write a fresh sheet at the end of the run.)

> **Important:** because we reload the workbook each time the program runs, repeated runs always start from a clean slate. That is usually what you want during testing. To persist a plan, add a `save_plan(...)` function that writes the results into a new sheet.

13. Putting It All Together

We have written every piece. Time to wire them up. Add this block at the bottom of `rig_planner.py` so the program runs three example jobs when you execute the file directly:

```
if __name__ == "__main__":
    db = RigDatabase("rig_database.xlsx")

    jobs = [
        Job("J-201", date(2026, 6, 1), duration_days=2,
            reamer_size_in=12.25, hole_length_m=1800),
        Job("J-202", date(2026, 6, 5), duration_days=3,
            reamer_size_in=17.5,  hole_length_m=1080),
        Job("J-203", date(2026, 6, 8), duration_days=1,
            reamer_size_in=8.5,   hole_length_m=900),
    ]

    plan = plan_jobs(db, jobs)
    print_plan(plan)
```

`print_plan` walks through the results and prints each attempt with a tick or dash so you can audit every decision at a glance:

```
def print_plan(results):
    print("=" * 72)
    print("RIG ASSIGNMENT PLAN")
    print("=" * 72)
    for job, winner, attempts in results:
        print(f"\nJOB {job.job_id}")
        print(f"  start={job.start_date}  days={job.duration_days}  "
              f"reamer={job.reamer_size_in}\"  hole={job.hole_length_m} m")
        for a in attempts:
            mark = "OK " if a.is_assignable else " - "
            print(f"  [{mark}] {a.rig_name}: {a.reason}")
        if winner:
            print(f"  >>> ASSIGNED to {winner.rig_name} "
                  f"({winner.rods_needed} x {winner.rod_size_in}\" rods)")
        else:
            print("  >>> NO RIG AVAILABLE")
```

13.1 Run it

```
python rig_planner.py
```

With our sample data you should see:

```
============================================================================
RIG ASSIGNMENT PLAN
============================================================================

JOB J-201
  start=2026-06-01  days=2  reamer=12.25"  hole=1800 m
  [ - ] RIG-A: [DATE FAIL] busy on 2026-06-01, 2026-06-02
  [OK ] RIG-B: PASS - 200 x 4.5" rods reserved (stock 250)
  >>> ASSIGNED to RIG-B (200 x 4.5" rods)

JOB J-202
  start=2026-06-05  days=3  reamer=17.5"  hole=1080 m
  [OK ] RIG-A: PASS - 120 x 5.5" rods reserved (stock 120)
  >>> ASSIGNED to RIG-A (120 x 5.5" rods)

JOB J-203
  start=2026-06-08  days=1  reamer=8.5"  hole=900 m
  [OK ] RIG-A: PASS - 100 x 3.5" rods reserved (stock 400)
  >>> ASSIGNED to RIG-A (100 x 3.5" rods)
============================================================================
```

14. Reading the Output

The output above is meant to read like a planner's notebook. Each block describes one job, lists every rig that was considered, and ends with the assignment. Let us go line by line.

14.1 Job J-201 - one rejection, one win

RIG-A was tried first and rejected at the date check: it is busy on both required days (2026-06-01 and 2026-06-02). Because we short-circuit, the program did not even look at RIG-A's reamer capability or rod inventory.

RIG-B was tried next. It is free on both days, it can run 12.25 in reamers, and we have 250 of the matching 4.5 in rods - more than the 200 the hole needs. **J-201 is assigned to RIG-B**, and stock of 4.5 in rods drops from 250 to 50.

14.2 Job J-202 - a tight rod check

J-202 starts 2026-06-05 for 3 days. RIG-A's busy set now only contains the original 2026-06-01..03 - it is free for J-202. The reamer check passes (RIG-A can run 17.5 in). The hole is 1 080 m, which needs `ceil(1080/9) = 120` rods of 5.5 in. Stock is exactly 120 - so the check just barely passes. **J-202 is assigned to RIG-A**, and 5.5 in stock drops to 0.

> **What if J-202 needed 121 rods?** The check would fail, the engine would try RIG-B (which cannot run a 17.5 in reamer at all - REAMER FAIL), then RIG-C, and so on. Try editing the hole length to 1 089 m and re-running to see this play out.

14.3 Job J-203 - small but easy

An 8.5 in reamer over 900 m. RIG-A is free that day (2026-06-08), is certified for 8.5 in, and we have 400 of the matching 3.5 in rods - we only need 100. **J-203 is assigned to RIG-A.**

14.4 A bird's-eye view

The final picture across all three jobs:

Job	Rig	Reamer	Rods used	Status
J-201	RIG-B	12.25 in	200 x 4.5 in	Assigned
J-202	RIG-A	17.5 in	120 x 5.5 in	Assigned
J-203	RIG-A	8.5 in	100 x 3.5 in	Assigned

15. Extending the System

Now that the foundation is in place, here are several practical extensions you may want to add. Each one slots into the existing code without changing the core checks.

15.1 Persist the plan back to Excel

Add a function that takes the `plan` list and writes a new sheet called `Plan` with one row per assignment. Use `openpyxl`:

```
from openpyxl import load_workbook

def save_plan(results, path: str = "rig_database.xlsx"):
    wb = load_workbook(path)
    if "Plan" in wb.sheetnames:
        del wb["Plan"]
    ws = wb.create_sheet("Plan")
    ws.append(["Job", "Rig", "StartDate", "Days",
               "Reamer_in", "Rods", "RodSize_in"])
    for job, winner, _ in results:
        if winner:
            ws.append([job.job_id, winner.rig_name,
                       job.start_date, job.duration_days,
                       job.reamer_size_in, winner.rods_needed,
                       winner.rod_size_in])
        else:
            ws.append([job.job_id, "UNASSIGNED",
                       job.start_date, job.duration_days,
                       job.reamer_size_in, 0, 0])
    wb.save(path)
```

15.2 Add priorities

Real jobs have a priority. Sort the job list by priority before running `plan_jobs` so the most critical jobs grab their rigs first:

```
jobs.sort(key=lambda j: j.priority, reverse=True)
plan = plan_jobs(db, jobs)
```

15.3 Consider crew availability

Add a fourth check (Step 4 - CREW) that verifies a certified crew is on roster for the entire duration. The pattern is identical to the date check - a set of busy days per crew - and it slots into `evaluate()` after the rod check.

15.4 Show alternatives, not just the first match

Right now we `break` after the first passing rig. To let the planner choose between options, remove the `break`, store ALL passing rigs, then pick the one with (say) the highest spare capacity or the closest location to the wellsite.

15.5 Run "what-if" scenarios

Because the database is just an Excel file, you can copy `rig_database.xlsx` to `rig_database_scenario_A.xlsx`, change a few values, then point the planner at it with `RigDatabase("rig_database_scenario_A.xlsx")`. Try doubling the rod stock, or marking a rig out of service, and see how the plan changes.

15.6 A graphical front-end

Tools like **Streamlit** turn a Python script into a web app with very little extra code. A 30-line Streamlit wrapper around `plan_jobs` gives non-programmers a browser UI to add jobs and view the schedule. That is a natural next milestone for this project.

> **One last tip:** as your rules grow, resist the urge to put more logic into the check functions. Instead, add new check functions and add one more line in `evaluate()`. Small, single-purpose functions stay easy to understand and easy to test - the most important property a planning tool can have.

Appendix A - Full create_database.py

This is the complete script that builds `rig_database.xlsx` from scratch. Run it once before running the planner.

```
"""
create_database.py
------------------
Builds the example Excel workbook used by the rig planning app.
The workbook contains four sheets:
    1. Rigs              - master list of rigs and capabilities
    2. RigSchedule       - which rig is busy on which dates
    3. ReamerCapability  - which reamer sizes each rig can run
    4. Rods              - rod inventory by size and availability
"""

from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from datetime import date

wb = Workbook()

# ---- Styling helpers ----
header_font = Font(name="Arial", bold=True, color="FFFFFF", size=11)
header_fill = PatternFill("solid", start_color="1F4E78")
center      = Alignment(horizontal="center", vertical="center")
thin        = Side(border_style="thin", color="BFBFBF")
border      = Border(left=thin, right=thin, top=thin, bottom=thin)


def style_header(ws, ncols):
    for col in range(1, ncols + 1):
        c = ws.cell(row=1, column=col)
        c.font = header_font
        c.fill = header_fill
        c.alignment = center
        c.border = border
    ws.row_dimensions[1].height = 22


# ============================================================================
# Sheet 1: Rigs
# ============================================================================
```

```
ws1 = wb.active
ws1.title = "Rigs"
ws1.append(["RigName", "Location", "MaxHoleLength_m", "Status"])
rigs = [
    ["RIG-A", "Field North", 3500, "Active"],
    ["RIG-B", "Field North", 2500, "Active"],
    ["RIG-C", "Field South", 4000, "Active"],
    ["RIG-D", "Field South", 1800, "Active"],
    ["RIG-E", "Field East",  3000, "Active"],
]
for r in rigs:
    ws1.append(r)
style_header(ws1, 4)
for col, w in zip("ABCD", [14, 14, 18, 12]):
    ws1.column_dimensions[col].width = w


# ============================================================================
# Sheet 2: RigSchedule - dates each rig is BUSY (unavailable)
# ============================================================================
ws2 = wb.create_sheet("RigSchedule")
ws2.append(["RigName", "BusyDate", "Job"])
schedule = [
    ["RIG-A", date(2026, 6, 1),  "Job-100"],
    ["RIG-A", date(2026, 6, 2),  "Job-100"],
    ["RIG-A", date(2026, 6, 3),  "Job-100"],
    ["RIG-B", date(2026, 6, 5),  "Job-101"],
    ["RIG-B", date(2026, 6, 6),  "Job-101"],
    ["RIG-C", date(2026, 6, 1),  "Job-102"],
    ["RIG-C", date(2026, 6, 10), "Job-103"],
    ["RIG-D", date(2026, 6, 8),  "Job-104"],
    ["RIG-E", date(2026, 6, 2),  "Job-105"],
    ["RIG-E", date(2026, 6, 3),  "Job-105"],
]
for r in schedule:
    ws2.append(r)
style_header(ws2, 3)
for col, w in zip("ABC", [14, 14, 14]):
```

```
    ws2.column_dimensions[col].width = w


# ============================================================================
# Sheet 3: ReamerCapability - which reamer sizes each rig can handle
# Sizes are in inches: 8.5, 12.25, 17.5
# ============================================================================
ws3 = wb.create_sheet("ReamerCapability")
ws3.append(["RigName", "ReamerSize_in"])
capability = [
    ["RIG-A", 8.5],
    ["RIG-A", 12.25],
    ["RIG-A", 17.5],   # RIG-A can do all three
    ["RIG-B", 8.5],
    ["RIG-B", 12.25],
    ["RIG-C", 12.25],
    ["RIG-C", 17.5],
    ["RIG-D", 8.5],
    ["RIG-E", 8.5],
    ["RIG-E", 12.25],
    ["RIG-E", 17.5],
]
for r in capability:
    ws3.append(r)
style_header(ws3, 2)
for col, w in zip("AB", [14, 16]):
    ws3.column_dimensions[col].width = w


# ============================================================================
# Sheet 4: Rods - inventory by size (length per rod = 9 m typical)
# ============================================================================
ws4 = wb.create_sheet("Rods")
ws4.append(["RodSize_in", "LengthPerRod_m", "AvailableCount"])
rods = [
    [3.5, 9, 400],   # standard drill pipe
    [4.5, 9, 250],   # heavyweight
    [5.5, 9, 120],   # large diameter
```

```
]
for r in rods:
    ws4.append(r)
style_header(ws4, 3)
for col, w in zip("ABC", [14, 18, 18]):
    ws4.column_dimensions[col].width = w


# ============================================================================
# Sheet 5: ReamerRodMapping - which rod size pairs with which reamer
# ============================================================================
ws5 = wb.create_sheet("ReamerRodMapping")
ws5.append(["ReamerSize_in", "RequiredRodSize_in"])
mapping = [
    [8.5,   3.5],
    [12.25, 4.5],
    [17.5,  5.5],
]
for r in mapping:
    ws5.append(r)
style_header(ws5, 2)
for col, w in zip("AB", [16, 20]):
    ws5.column_dimensions[col].width = w


out = "/home/claude/rig_planner/rig_database.xlsx"
wb.save(out)
print(f"Saved: {out}")
```

Appendix B - Full rig_planner.py

This is the complete planning program. Save it as `rig_planner.py` in the same folder as the Excel file, then run it with `python rig_planner.py`.

```
"""
rig_planner.py
==============
A small planning app that decides which rig can take a job by running
THREE sequential checks against an Excel database:

    Step 1 - DATE         : is the rig free on every day the job needs?
    Step 2 - REAMER SIZE  : can the rig physically run that reamer?
    Step 3 - RODS         : do we own enough rods (right size) for the hole?

The checks are *short-circuiting*: as soon as one fails, the rig is
rejected and the next checks are skipped (this matches the user's
"if one got crossed, dont check the rest" rule).

Run:
    python rig_planner.py
"""

from __future__ import annotations
from dataclasses import dataclass
from datetime import date, timedelta
from typing import List, Dict, Set, Tuple
import math
import pandas as pd

DB_PATH = "rig_database.xlsx"


# ---------------------------------------------------------------------------
# Data classes - lightweight "record" types so code reads like English.
# ---------------------------------------------------------------------------
@dataclass
class Job:
    """A single job we want to plan."""
    job_id: str
    start_date: date
    duration_days: int
    reamer_size_in: float
```

```
    hole_length_m: float

    @property
    def required_dates(self) -> List[date]:
        """Every calendar day the rig is needed for this job."""
        return [self.start_date + timedelta(days=i)
                for i in range(self.duration_days)]


@dataclass
class PlanResult:
    """Outcome of trying to assign one job to one rig."""
    job_id: str
    rig_name: str
    passed_date: bool = False
    passed_reamer: bool = False
    passed_rods: bool = False
    rods_needed: int = 0
    rod_size_in: float = 0.0
    reason: str = ""

    @property
    def is_assignable(self) -> bool:
        return self.passed_date and self.passed_reamer and self.passed_rods


# ---------------------------------------------------------------------
# Database loader - reads every sheet of the workbook into pandas
# DataFrames, then converts them to fast lookup structures.
# ---------------------------------------------------------------------
class RigDatabase:
    def __init__(self, path: str = DB_PATH):
        self.path = path
        sheets = pd.read_excel(path, sheet_name=None)

        self.rigs_df         = sheets["Rigs"]
        self.schedule_df     = sheets["RigSchedule"]
        self.capability_df   = sheets["ReamerCapability"]
```

```
        self.rods_df         = sheets["Rods"]
        self.mapping_df      = sheets["ReamerRodMapping"]

        # Normalise the date column to plain `date` objects.
        self.schedule_df["BusyDate"] = pd.to_datetime(
            self.schedule_df["BusyDate"]).dt.date

        # ---- Pre-build lookups ----
        # 1) Busy days per rig: {"RIG-A": {date, date, ...}}
        self.busy: Dict[str, Set[date]] = {}
        for rig in self.rigs_df["RigName"]:
            self.busy[rig] = set(
                self.schedule_df.loc[
                    self.schedule_df["RigName"] == rig, "BusyDate"])

        # 2) Reamer capability per rig: {"RIG-A": {8.5, 12.25, 17.5}}
        self.capable: Dict[str, Set[float]] = {}
        for rig in self.rigs_df["RigName"]:
            self.capable[rig] = set(
                self.capability_df.loc[
                    self.capability_df["RigName"] == rig,
                    "ReamerSize_in"].astype(float))

        # 3) Reamer -> rod size map: {8.5: 3.5, 12.25: 4.5, 17.5: 5.5}
        self.reamer_to_rod: Dict[float, float] = dict(
            zip(self.mapping_df["ReamerSize_in"].astype(float),
                self.mapping_df["RequiredRodSize_in"].astype(float)))

        # 4) Rod inventory keyed by size:
        #    {3.5: {"length_per_rod": 9, "available": 400}}
        self.rod_inventory: Dict[float, Dict[str, float]] = {}
        for _, row in self.rods_df.iterrows():
            self.rod_inventory[float(row["RodSize_in"])] = {
                "length_per_rod": float(row["LengthPerRod_m"]),
                "available":      int(row["AvailableCount"]),
            }

    # ----- Convenience -----
```

```
    @property
    def rig_names(self) -> List[str]:
        return list(self.rigs_df["RigName"])

    def max_hole_length(self, rig: str) -> float:
        return float(self.rigs_df.loc[
            self.rigs_df["RigName"] == rig, "MaxHoleLength_m"].iloc[0])


# ---------------------------------------------------------------------
# The three sequential checks - each returns (passed, reason).
# ---------------------------------------------------------------------
def check_date(db: RigDatabase, rig: str, job: Job) -> Tuple[bool, str]:
    """Step 1: rig must be free on every day in job.required_dates."""
    clashes = [d for d in job.required_dates if d in db.busy[rig]]
    if clashes:
        return False, f"busy on {', '.join(str(d) for d in clashes)}"
    return True, "all required dates free"


def check_reamer(db: RigDatabase, rig: str, job: Job) -> Tuple[bool, str]:
    """Step 2: rig must be able to run the requested reamer size."""
    if job.reamer_size_in not in db.capable[rig]:
        return False, (f"cannot run {job.reamer_size_in}\" reamer "
                       f"(capable: {sorted(db.capable[rig])})")
    return True, f"{job.reamer_size_in}\" reamer supported"


def check_rods(db: RigDatabase, rig: str, job: Job
               ) -> Tuple[bool, str, int, float]:
    """Step 3: enough rods of the right size for the hole length?
    Also enforces the rig's max hole length."""
    # 3a) rig's own depth limit
    if job.hole_length_m > db.max_hole_length(rig):
        return (False,
                f"hole length {job.hole_length_m} m exceeds rig limit "
                f"{db.max_hole_length(rig)} m",
                0, 0.0)
```

```
    # 3b) which rod pairs with this reamer?
    rod_size = db.reamer_to_rod.get(job.reamer_size_in)
    if rod_size is None:
        return False, f"no rod mapping for reamer {job.reamer_size_in}\"", 0, 0.0

    inv = db.rod_inventory.get(rod_size)
    if inv is None:
        return False, f"rod size {rod_size}\" not in inventory", 0, rod_size

    # 3c) how many rods does this hole need? Always round up.
    rods_needed = math.ceil(job.hole_length_m / inv["length_per_rod"])

    if rods_needed > inv["available"]:
        return (False,
                f"need {rods_needed} x {rod_size}\" rods, "
                f"only {inv['available']} available",
                rods_needed, rod_size)

    return (True,
            f"{rods_needed} x {rod_size}\" rods reserved "
            f"(stock {inv['available']})",
            rods_needed, rod_size)


# ---------------------------------------------------------------------
# Engine - runs the three checks SEQUENTIALLY and short-circuits.
# ---------------------------------------------------------------------
def evaluate(db: RigDatabase, rig: str, job: Job) -> PlanResult:
    res = PlanResult(job_id=job.job_id, rig_name=rig)

    # Step 1 ------------------------------------------------------------
    ok, why = check_date(db, rig, job)
    res.passed_date = ok
    if not ok:
        res.reason = f"[DATE FAIL] {why}"
        return res                      # short-circuit, skip 2 & 3
```

```
    # Step 2 ------------------------------------------------------
    ok, why = check_reamer(db, rig, job)
    res.passed_reamer = ok
    if not ok:
        res.reason = f"[REAMER FAIL] {why}"
        return res                      # short-circuit, skip 3

    # Step 3 ------------------------------------------------------
    ok, why, n_rods, rod_size = check_rods(db, rig, job)
    res.passed_rods   = ok
    res.rods_needed   = n_rods
    res.rod_size_in   = rod_size
    res.reason        = ("PASS - " + why) if ok else f"[ROD FAIL] {why}"
    return res


# ---------------------------------------------------------------------
# Greedy assignment: for each job, pick the first rig that passes all
# three checks, then reserve its dates & rods so later jobs see the
# updated state.
# ---------------------------------------------------------------------
def plan_jobs(db: RigDatabase, jobs: List[Job]
              ) -> List[Tuple[Job, PlanResult | None, List[PlanResult]]]:
    """Returns a list of tuples: (job, winning_result_or_None, all_attempts)."""
    output = []
    for job in jobs:
        attempts: List[PlanResult] = []
        winner: PlanResult | None = None

        for rig in db.rig_names:
            r = evaluate(db, rig, job)
            attempts.append(r)
            if r.is_assignable:
                winner = r
                # ---- reserve resources so the next job sees them used ----
                for d in job.required_dates:
                    db.busy[rig].add(d)
                db.rod_inventory[r.rod_size_in]["available"] -= r.rods_needed
```

```
                break

        output.append((job, winner, attempts))
    return output


# ---------------------------------------------------------------------
# Pretty printing
# ---------------------------------------------------------------------
def print_plan(results) -> None:
    print("=" * 72)
    print("RIG ASSIGNMENT PLAN")
    print("=" * 72)
    for job, winner, attempts in results:
        print(f"\nJOB {job.job_id}")
        print(f"  start={job.start_date}  days={job.duration_days}  "
              f"reamer={job.reamer_size_in}\"  hole={job.hole_length_m} m")
        for a in attempts:
            mark = "OK " if a.is_assignable else " - "
            print(f"  [{mark}] {a.rig_name}: {a.reason}")
        if winner:
            print(f"  >>> ASSIGNED to {winner.rig_name} "
                  f"({winner.rods_needed} x {winner.rod_size_in}\" rods)")
        else:
            print("  >>> NO RIG AVAILABLE")
    print("\n" + "=" * 72)


# ---------------------------------------------------------------------
# Demo: three jobs (matches the example in the user's request)
# ---------------------------------------------------------------------
if __name__ == "__main__":
    db = RigDatabase(DB_PATH)

    jobs = [
        Job("J-201", date(2026, 6, 1), duration_days=2,
            reamer_size_in=12.25, hole_length_m=1800),
        Job("J-202", date(2026, 6, 5), duration_days=3,
            reamer_size_in=17.5,  hole_length_m=1080),
        Job("J-203", date(2026, 6, 8), duration_days=1,
            reamer_size_in=8.5,   hole_length_m=900),
    ]

    plan = plan_jobs(db, jobs)
    print_plan(plan)
```

Appendix C - Glossary

Term	Meaning
DataFrame	A table-like object provided by pandas. Has named columns and a row index.
dataclass	A Python class decorator that auto-generates __init__ and __repr__ from field declarations.
dict (dictionary)	A Python container that maps keys to values. Lookups are constant time.
set	A Python container of unique values. Membership tests ("is X in here?") are constant time.
normalised	A database design where each fact lives in exactly one place, avoiding duplication.
short-circuit	Stop evaluating as soon as the answer is known. Our engine stops checking a rig as soon as one che
greedy algorithm	Make the locally best choice at each step. Simple, fast, often good enough.
virtual environment	A private Python copy for one project, so libraries do not collide across projects.
openpyxl	The Python library that actually reads and writes .xlsx files. pandas uses it under the hood.
reamer	Drilling tool that enlarges an existing hole. Sized in inches; bigger reamer means bigger hole.
rod	A length of drill pipe. Stacked end-to-end to reach the target depth.

End of tutorial. You now have a working Python planning app backed by an Excel database, organised around three sequential, short-circuiting checks. Happy planning.